Critical Stations and Live Disruptions in the New York City Subway

Master Data and Intelligence for Smart Systems Data Processing and Analytics Project

Made By: Oussama Sbaa / Mazen Morsy / Abdullah Hameed

GitHub repository link : https://github.com/oussama-sbaa1/DPA_Project_NYC_SUBWAY

1/ Domain , Overarching Question and Part 1 Sub-Question:

A. **Domain:** Public transport, the New York City subway: critical stations and live disruptions
B. **Overarching Question :** Which subway stations are critical for the network, and are live disruptions happening in those stations or not?
C. **Part 1-Volume (Spark SQL) Sub-question:** What is the usual ridership in each subway station by hour and day of the week, and by how much does the weather change it?

2/ Datasets sources for each part: links, sizes and licenses. How the data is joined for part 1 and Why Spark ? :

A. Datasets source for Part 1 : links to the sources, sizes and licenses

Dataset	Link	Size of what we take	Licence
MTA Subway Hourly Ridership: 2020–2024	data.ny.gov/d/wujg-7c2s	Jul–Dec 2024 **14.0 M rows · 2.43 GB**	OPEN-NY
MTA Subway Hourly Ridership: Beginning 2025	data.ny.gov/d/5wq4-mkjj	Jan–Jun 2025 **15.4 M rows · 2.65 GB**	OPEN-NY
NOAA Local Climatological Data v2, Central Park (USW00094728)	ncei.noaa.gov (LCD v2 API)	Hourly, Jul 2024–Jun 2025 **11,765 rows · 5.7 MB**	U.S. public domain, cite NOAA
MTA Subway Stations (links complexes, stop IDs and GTFS stops)	data.ny.gov/d/39hk-dx4f	Whole table **496 rows · 69 KB**	OPEN-NY

Part 1 volume: 29.4 M rows · 5.08 GB of CSV for 12 months . Raw files on the original database should be kept untouched According to the licensing rules for MTA database regulations, therefore we created our own copies to manipulate. For that every change is written to new parquet tables.

B. How the data is joined for Part :

There are two real joins in our datasets:

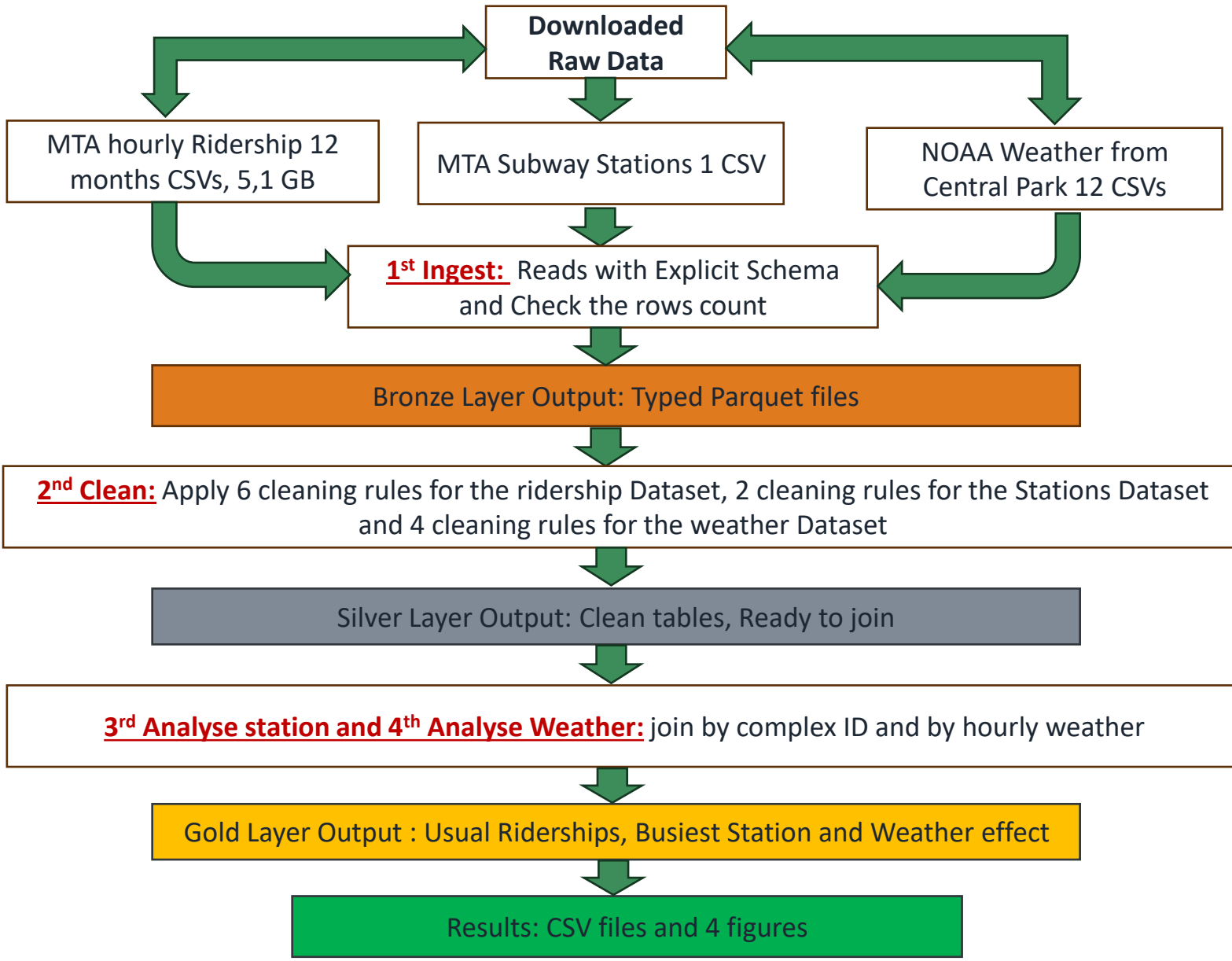
i. **Ridership dataset with stations dataset:** We joined by Complex_ID so that each ridership database row gets also the stations name and the borough. This is done in order to know about the busiest stations and to compare boroughs.
ii. **Ridership dataset with weather dataset:** We joined the hour (New York time). Each hour of ridership gets its corresponding weather (dry, light rain, heavy rain, snow). This is done to be able to compare rides in rainy or snowy hours with the rides in dry hours.

C. Why we chose to use Spark :

We have mainly used spark because of the Size of Data we have. In fact, Ridership data alone is 5.08 GB of CSVs. This data size is too big for normal tools. Matter of fact 1 month data is already twice Excel's limit of 1,048,576 rows. And if we use Pandas dataframes, it would have to load the whole table into the laptop's memory at once and overloading it. The interesting part of using Spark here is its ability to process large datasets by splitting data into small pieces and processing them in parallel on all the processor cores which results in the cleaning of 29.4 million rows in a matter of few minutes.

3/ Pipeline: a diagram and a description of the analysis process from raw data to results

A. Pipeline Diagram: Divided into Layers: **N.B** All steps run in Docker (Jupyter + Spark 3.5.0)



B. A description of the analysis process from raw data to results:

Download → Raw: 00_download notebook: it downloads the ridership (12 files), stations (1 file) and weather (12 files) as CSV, the whole data sets are about 5.1 GB.
Ingest → Bronze Layer: 01_ingest notebook: it reads the CSV files with an explicit schema and saves them as typed Parquet.
Clean → Silver Layer: 02_clean notebook: it applies all of the 12 cleaning rules (6 for the riderships dataset, 2 for the stations dataset and 4 for the weather dataset) and results in 3 clean tables ready to join.
Analysis → Gold Layer: 03_analyze_stations notebook: it computes the usual ridership per station and the busiest stations.
04_analyze_weather notebook: it compares rainy and snowy hours with dry weather hours.
Results: we end up with CSV tables and also 4 figures.

4/ Data cleaning. what we found, what we did, row counts before/after

A. Data Cleaning and what we have found:

- The ridership dataset contains also tramway and railway stations data which we will not need. Also, The Station Complex ID is considered as a string but it is an integer in stations dataset. In addition, We found 1250 rows of data for an hour that doesn't exist because of the switch to the summer hour on the 9th of march 2025 at 2:00. Finally, we found that MTA writes multiple rows for the same station at the same hour.
- In the Stations dataset, we found that the boroughs are written by codes (M, BK etc..) which doesn't correlate with the ridership datasets because it has full names for the boroughs. Also, we found several rows per station complex
- In the Weather dataset we have extra reports about the weather. There were also lines with very negligible rain value <0.1 mm due to the weather measurement errors noted as "T". In addition, we found that we have 317 hours without any rain value Finally, the time in this datasets does not change accordingly because usually New York time moves by 1 hour in summer time.

B. How did we proceed (what we did about the findings in data):

- For the ridership dataset: We removed Tramway and Railway rows, we converted the complex ID to a number, we removed those 1250 rows for the hour that doesn't exist and finally we added up in order to obtain one row per station complex and hour.
- For the stations dataset: we replaced boroughs codes with full corresponding names and we grouped into one row per station complex instead of many rows.
- For the weather dataset: we kept only the hourly weather reports and dropped others. We moved ahead with 1 hour for the summer time. We replaced the trace of rain T with 0. We also have merged to obtain one row per hour.

C. The row counts before/after:

Table	Step	Rows_Before	Rows After
Ridership	R1 : keep the subway only	29375687	29145131
Ridership	R2 : drop missing or negative values	29145131	29145131
Ridership	R3 :station id text -> integer	29145131	29145131
Ridership	R4 : drop the hour that does not exist (2025-03-09 02:00)	29145131	29143881
Ridership	R5 : drop duplicate rows	29143881	29143881
Ridership	R6 : one row per station and hour	29143881	3620146
Stations	S2 : one row per station complex	496	445
Weather	W1 : keep the hourly reports FM-15	11765	8771
Weather	W4 : one row per New York hour	8771	8704

5/ Important design decisions

1. Aggregate ridership by station complex and hour

- The raw MTA data contains multiple records for the same station complex and hour, including different fare categories. Summing those records produces one ridership value per station complex per hour.

2. The use of dry weather as the baseline

- Dry hours represent the reference condition against which rainy and snowy hours are compared . To make this comparison fairer, we calculated usual ridership for matching month, day of the week, and hour of the day

3. Excluding records with missing weather conditions

- The weather table has 317 records with a missing `condition`, leaving 8,387 records with known conditions. We excluded those records from the weather-effect analysis because we could not classify them as dry, light rain, heavy rain, or snow.

6/ The answer to our part 1 Sub-question

The Sub-question: What is the usual ridership in each subway station by hour and day of the week, and by how much does the weather change it?. **Answer :**

1. Network-wide effect of weather

Weather condition	Hours observed	Ridership change vs baseline
Light rain	454	−2.0%
Heavy rain	110	−1.0%
Snow	64	−4.8%

Snow causes the largest estimated network-wide decrease, at 4.8%

2. Differences between boroughs

Borough	Light rain	Heavy rain	Snow
Bronx	−3.7%	−10.3%	−9.0%
Brooklyn	−4.1%	−5.7%	−8.7%
Manhattan	−0.3%	+3.2%	−2.1%
Queens	−4.2%	−7.1%	−8.6%

Manhattan behaved differently: its estimated ridership was slightly higher during heavy rain (3.2%) and almost unchanged during light rain (−0.3%). This suggests weather related ridership patterns vary geographically rather than affecting every borough equally